

HealthPro — Моє Здоров'я

Детальний звіт стану проєкту та план подальших дій

Версія звіту: 2.0 | Дата: 25 квітня 2026 р. | Підготовлено: Claude Sonnet (Anthropic)

ВИКОНАВЧЕ РЕЗЮМЕ

Проєкт HealthPro пройшов суттєвий рефакторинг (Етапи 4-A — 4-Д): монолітний файл `src/app.js` зменшено з 1879 до 254 рядків, створено 34 модульних файли у 8 доменних директоріях. Архітектура event-driven рухається в правильному напрямку. Проте перед переходом до Етапу 5 (SQLite) та нативної збірки Capacitor існує ряд критичних невирішених проблем, що можуть заблокувати подальший розвиток.

ЗРОБЛЕНО ДОБРЕ

- Модульність Етапу 4 завершено
- Event-driven шина `on/emit`
- i18n структура (`uk`, `ru`, `pdf`)
- `storage.js` з міграцією `LS` → `IDB`
- `app.js`: 254 рядки (було 1879)

ПОТРЕБУЄ УВАГИ

- `utils.js` майже порожній (8 рядків)
- `setLang()` ще залишається в `app.js`
- 1 CSS-файл замість 7 за планом
- `constants.js` не створено
- `i18n/index.js` не завершено

КРИТИЧНІ ПРОБЛЕМИ

- `node_modules` закомічено у репо
- Типо `HealtPro` — не виправлено
- `core/platform.js` відсутній
- `capacitor webDir` = `Windows`-шлях
- FOUC-баг активний

1. ПОТОЧНИЙ СТАН КОМПОНЕНТІВ

Компонент	Деталі та поточний стан	Статус
Vite + bundler	<code>vite.config.js</code> налаштовано, порт 5000, HMR вимкнено	Готово
<code>src/app.js</code>	254 рядки (було 1879) — оркестратор подій	Готово
Feature-модулів	8 доменів: <code>pressure</code> , <code>charts</code> , <code>meds</code> , <code>steps</code> , <code>analytics</code> , <code>history</code> , <code>export</code> , <code>settings</code> , <code>pwa</code> — 34 файли	Готово
<code>core/state.js</code>	63 рядки, шина <code>on/emit</code> , <code>setToast</code>	Готово
<code>core/storage.js</code>	168 рядків, міграція <code>LS</code> → <code>IDB</code> реалізована	Готово

i18n/ uk, ru, pdf	Файли словників є. setLang() та t() ще в app.js	~70%
core/utils.js	Лише 8 рядків — форматери та helpers відсутні	~30%
styles/	Один файл app.css (509 рядків) замість 7 за планом	~50%
core/constants.js	Файл не створено. Магічні рядки розкидані по модулях	Відсутній
core/platform.js	НЕ СТВОРЕНО. Критичний шар для переходу на Capacitor	Відсутній
capacitor.config	webDir вказує на Windows-шлях замість 'dist'	Помилка
Capacitor-плагіни	Жодного плагіна у package.json (notifications, share...)	Відсутні
node_modules/	Закомічено у репо (~100-200 МБ). Гальмує репо	Критично
Типо HealtPro	Папка HealtPro замість HealthPro. Не виправлено	Критично
Тести (vitest)	Жодного тесту. getBPNorm, calcScore — не перевірено	Відсутні
FOUC-баг	CSS-спалах 1-2с при кожному відкритті сторінки	Активний

2. ПРОГРЕС ПО ФАЗАХ ROADMAP

Фаза	Назва	Прогрес	Коментар
Фаза 0	Прибирання репо	40%	node_modules та типо — не виправлено
Фаза 1	Модульна архітектура	70%	app.js готовий, але platform.js, constants.js, utils.js — не завершено
Фаза 2	Capacitor + Android APK	5%	Конфіг є з помилкою webDir, плагінів немає
Фаза 3	Якість, тести, безпека	0%	Жодного тесту, жодної перевірки безпеки
Етап 5	SQLite база даних	0%	Не починати поки Фаза 1 не завершена на 90%+

Ключовий принцип: Фаза 1 має бути завершена на 90%+ перед початком Фази 2 (Capacitor APK) або Етапу 5 (SQLite). Пропуск кроків накопичує технічний борг і блокує розвиток.

3. КРИТИЧНІ ПРОБЛЕМИ ТА РИЗИКИ

Ризик	Детальний опис проблеми	Пріоритет
FOUC-баг	При кожному відкритті сторінки 1-2 секунди видно незастилений білий HTML, а потім різко з'являється темний UI. Причина: браузер рендерить HTML до завантаження зовнішнього CSS-файлу.	КРИТИЧНО
node_modules/	Директорія node_modules закомічена у Git-репозиторій (~100-200 МБ). Кожен git push/pull повільніший, PR-рев'ю нечитабельне, Replit завантажується довше.	КРИТИЧНО
Тypo HealtPro	Папка називається HealtPro замість HealthPro (пропущена буква h). Зараз виправлення займе 1 годину. Через місяць, коли додасться ще 50+ модулів — це буде повний день роботи.	КРИТИЧНО
platform.js	Шар абстракції web vs Capacitor відсутній. Без нього кожен модуль, що торкається нотифікацій, файлів або Share, доведеться переписувати двічі при переході на native.	ВИСОКИЙ
capacitor webDir	У capacitor.config.json параметр webDir містить Windows-шлях замість відносного 'dist'. Білд Android APK завалиться на першій же спробі прх cap sync.	ВИСОКИЙ
MySQL план	MySQL вимагає бекенду, REST API, GDPR-відповідності, SSL/TLS, резервного копіювання сервера. Для медичного PWA це окремий проєкт на 2-3 місяці розробки. SQLite вирішує все локально.	ВИСОКИЙ
Тести відсутні	Функції getBPNorm, calcHealthScore, calcBMI не мають жодного тесту. Помилка в цих функціях означає невірний медичний висновок для користувача. Це не технічний борг — це ризик для здоров'я.	ВАЖЛИВО
constants.js	Магічні рядки (ключі localStorage, версія дисклеймера, пороги) розкидані по різних модулях. Рефакторинг без централізованих констант є джерелом прихованих багів.	ВАЖЛИВО

3.1 FOUC-баг — швидке виправлення

Додати один рядок до <head> в index.html перед тегом <link> CSS:

```
<style>html,body{background:#080d1a!important;color:#e2e8f0!important}</style>
```

Це фарбує фон миттєво, до завантаження зовнішнього CSS. Час виправлення: 15 хвилин.

3.2 Видалення node_modules з репозиторію

Виконати три команди у терміналі:

```
echo "node_modules/" >> .gitignore
git rm -r --cached node_modules
git commit -m "chore: remove node_modules, add .gitignore"
```

3.3 Виправлення тупо у назві папки

Перейменувати HealtPro-Moie-Zdorovia → HealthPro-Moie-Zdorovia, потім оновити рядок args у файлі .replit та перевірити всі import-шляхи. Зараз це 1 година роботи — через місяць буде повний день.

4. MySQL vs SQLite — ПРАВИЛЬНИЙ ВИБІР БАЗИ ДАНИХ

У плані підготовки до Етапу 5 згадується MySQL. Для медичного PWA-застосунку це рішення потребує детального обґрунтування, оскільки вибір бази даних безпосередньо впливає на безпеку медичних даних.

Критерій	SQLite (рекомендовано)	MySQL (передчасно)
Розміщення	На пристрої користувача	На зовнішньому сервері
Офлайн-робота	Повна підтримка, завжди	Відсутня — потрібен інтернет
Для Capacitor	Нативна підтримка плагіном	Потребує REST API та бекенду
Медичні дані	Локально на пристрої — безпечно	Сервер = GDPR + відповідальність
Складність	Низька — один плагін	Висока: сервер, auth, SSL/TLS
Ліміт даних	До 2 ГБ, шифрування AES-256	Необмежено, але потрібен DevOps
Вартість	Безкоштовно	Хостинг + підтримка сервера
Готовність	ПРАВИЛЬНИЙ ВИБІР ДЛЯ ЗАРАЗ	ОКРЕМИЙ ПРОЕКТ — не зараз

ВАЖЛИВО: HealthPro обробляє медичні дані (артеріальний тиск, пульс, список ліків, особисті дані користувача). Зберігання цих даних на зовнішньому сервері (MySQL) автоматично вимагає: шифрування каналу SSL/TLS, шифрування даних at rest, систему автентифікації та авторизації, відповідність вимогам GDPR, резервне копіювання сервера та моніторинг. Це окремий проєкт обсягом 2-3 місяці розробки. SQLite на пристрої вирішує всі ці питання автоматично і безкоштовно.

5. ПОКРОКОВИЙ ПЛАН ДІЙ

Кроки 1-4 необхідно виконати негайно, до будь-яких інших змін у коді. Вони усувають проблеми, які блокують подальший розвиток.

#	Дія	Орієнтовний час	Стан
1	Видалити node_modules з репо: git rm --cached node_modules, додати .gitignore	30 хвилин	Не зроблено

2	Виправити capacitor.config.json: змінити webDir на 'dist'	5 хвилин	Не зроблено
3	Виправити typo: HealtPro → HealthPro, оновити .replit та шляхи	1-2 години	Не зроблено
4	Додати inline critical CSS у для усунення FOUC-бага	15 хвилин	Не зроблено
5	Створити core/constants.js з усіма ключами та константами	2 години	Заплановано
6	Наповнити core/utils.js: formatTime, formatDate, avg та інші helpers	2 години	Заплановано
7	Створити core/platform.js з web-fallback для нотифікацій, файлів, share	1 день	Заплановано
8	Перенести setLang() та t() з app.js до i18n/index.js	2 години	Заплановано
9	Розбити styles/app.css на 7 файлів згідно плану архітектури	3 години	Заплановано
10	Написати тести Vitest: getBPNorm, calcBMI, calcHealthScore	1 день	Заплановано
11	Встановити @capacitor-community/sqlite (замість MySQL)	Етап 5	В планах
12	Створити core/db/sqlite.js: open, migrate, runQuery, getRows	Етап 5	В планах
13	Міграція localStorage → SQLite з JSON-резервною копією	Етап 5	В планах
14	Перший Android APK: npx cap build android → тест на реальному пристрої	Фаза 2	В планах

6. ДЕТАЛЬНИЙ ПЛАН ЕТАПУ 5 — SQLite

Розпочинати тільки після завершення Фази 1 на 90%+. Обов'язкова передумова: core/platform.js створений і протестований.

5-А	Встановити @capacitor-community/sqlite Команда: npm install @capacitor-community/sqlite. Створити файл core/db/sqlite.js з методами: open(), migrate(), runQuery(), getRows(). Web-fallback реалізується через sql.js (IndexedDB) — той самий API працює і у браузері, і у native Capacitor.
5-Б	Схема бази даних Таблиці: measurements (id, ts, sys, dia, pulse, note), pills (id, name, dose, time, days, date), pills_taken (date, pill_id, taken), settings (key, value). Обов'язковий індекс: idx_measurements_ts ON measurements(ts DESC) для швидкої вибірки за датою.
5-В	Міграція localStorage → SQLite Одноразова міграція з прогрес-індикатором у UI. ОБОВ'ЯЗКОВО зробити JSON-резервну копію перед початком міграції. Після переносу перевірити цілісність усіх даних (кількість записів, контрольні суми).
5-Г	Оновити модулі для роботи з БД Модулі pressure/, meds/, history/, analytics/ — замінити читання з масивів state на SQL-запити до БД. Зберегти сумісність публічного API для залежних модулів. State стає read-only after-load.
5-Д	Резервне копіювання та шифрування Реалізувати експорт/імпорт файлу .db через core/platform.js. Опціональне шифрування AES-256 для захисту медичних даних. Інтеграція з Google Drive та iCloud через @capacitor/filesystem.

7. ЦІЛЬОВА АРХІТЕКТУРА ПІСЛЯ РЕФАКТОРИНГУ

Повна структура src/ після завершення всіх фаз:

```
src/
  app.js           оркестратор (254р – готово)
  core/
    state.js       стан + on/emit + setToast (готово)
    storage.js     LS-обгортка / міст до SQLite (готово)
    utils.js       formatTime, avg, helpers (доробити)
    constants.js   DISCLAIMER_VERSION, ключі (ВІДСУТНІЙ)
    platform.js    web vs Capacitor abstraction (ВІДСУТНІЙ)
    db/
      sqlite.js    open, migrate, runQuery (Етап 5)
  i18n/
```

index.js	setLang, t, renderWelcome (доробити)
ui.uk.js	словник UA (готово)
ui.ru.js	словник RU (готово)
pdf.js	PDF-мітки (готово)
features/	
pressure/	norm, who, critical, index (готово)
charts/	helpers, bp-chart, index (готово)
meds/	drug-db, index (готово)
steps/	index (готово)
analytics/	health-score, bmi, reco, trend (готово)
history/	index (готово)
export/	csv, modal, pdf, print, logo (готово)
settings/	theme, i18n, profile, notif (готово)
pwa/	index (готово)
tests/	
norm.test.js	getBPNorm — ОБОВ'ЯЗКОВО
bmi.test.js	calcBMI, getBMICategory
score.test.js	calcHealthScore

8. ВИСНОВКИ ТА РЕКОМЕНДАЦІЇ

Сильні сторони проєкту

Рефакторинг Етапу 4 виконано якісно і системно. Архітектура event-driven з шиною on/emit є правильним технічним рішенням для PWA. Файл storage.js з міграцією LS → IDB — відмінна підготовка до переходу на SQLite. app.js зменшено у 7 разів: з 1879 до 254 рядків. Проєкт рухається в правильному напрямку.

Що необхідно завершити перед Етапом 5

Виправити 4 критичні проблеми: node_modules у репо, typo HealtPro, відсутній platform.js, некоректний capactor webDir. Без platform.js перехід на Capactor перетвориться на хаос подвійного переписування. Без виправлення typo рефакторинг зламає всі import-шляхи.

Чого НЕ робити

НЕ починати MySQL — це передчасно і потенційно небезпечно для медичних даних. НЕ пропускати тести для getBPNorm та calcHealthScore: помилка в цих функціях означає невірний медичний висновок для кінцевого користувача. НЕ переходити на Фазу 2 поки Фаза 1 не завершена на 90%+.

Наступні 4 кроки (виконати зараз — до 2 годин роботи)

1. git rm -r --cached node_modules та додати .gitignore.
2. capacitor.config.json: виправити webDir: 'dist'.
3. Перейменувати HealtPro → HealthPro, оновити .replit.
4. Додати inline critical CSS у head для усунення FOUC-бара.